Conducción autónoma

Leire Gonzalez, Aroa Gomez, Udane Saitua

Diciembre 2025

1 Introduction

El objetivo principal de esta práctica es desarrollar un robot con capacidad de conducción autónoma, capaz de navegar de manera segura y eficiente por un entorno controlado. Para lograrlo, el robot debe cumplir tres tareas fundamentales: seguir correctamente la carretera, detectar y reconocer señales de tráfico, y actuar de acuerdo con las señales y situaciones de intersección que se presenten a lo largo del recorrido.

El entorno de prueba se ha implementado inicialmente en simulación mediante Gazebo, utilizando el escenario yahboom track signs, y controlado a través de ROS2. Esta configuración permite verificar el comportamiento del robot antes de aplicarlo en un entorno real, asegurando que los algoritmos de percepción y control funcionen correctamente.

El robot puede encontrarse con cuatro tipos de señales: STOP, Giro a la izquierda, Giro a la derecha y Seguir recto. Cada señal requiere una acción específica, que puede incluir detenerse, girar o continuar en línea recta.

Para abordar este problema, se ha diseñado una arquitectura de control basada en la subsunción, en la que distintos subcomportamientos se coordinan para tomar decisiones en tiempo real. Esta arquitectura permite que el robot priorice comportamientos críticos, como la respuesta ante señales de STOP, mientras mantiene otras funciones como el seguimiento de la carretera activas.

El presente informe documenta el diseño y la implementación de esta arquitectura, la estrategia de control adoptada, los resultados de la experimentación en simulación, y las conclusiones y posibles mejoras que se podrían implementar en el futuro.

Además, el problema de la conducción autónoma presenta diversos retos, como la necesidad de reaccionar en tiempo real ante estímulos visuales, gestionar información potencialmente ruidosa (de los sensores) y coordinar distintas tareas con prioridades diferentes. Por este motivo, resulta fundamental diseñar una arquitectura de control que sea robusta y fácilmente ampliable, permitiendo que el sistema responda de forma adecuada incluso ante situaciones imprevistas.

2 Estrategia

Para el control del robot autónomo, se ha adoptado una arquitectura de subsunción, que permite coordinar distintos subcomportamientos de manera jerárquica según su prioridad.

2.1 Subcomportamientos

El problema de conducción autónoma se ha descompuesto en los siguientes subcomportamientos:

2.1.1 Seguimiento de la carretera (Road Following)

Este subcomportamiento tiene como objetivo mantener al robot centrado en la carretera y avanzar de forma estable, evitando desviaciones que puedan provocar que se salga del carril o choque contra los bordes del entorno. Es el comportamiento base sobre el que se construyen los demás subcomportamientos (giros y detención).

Para poder realizar esto nuestra estrategia ha sido encontrar el punto que tenga el obstáculo mas lejos; es decir, buscar la zona más profunda o con más espacio libre desde la perspectiva del robot y seguir esa dirección.
Para llevar a cabo esta estrtegia, la parte fundamental del algoritmo es el procesamiento de la imagen. A la imagen recibida se le aplica una máscara dónde blanco (255,255,255) representa "carretera" y negro (0,0,0) "no carretera". El objetivo es encontrar la columna vertical con mayor número de blancos, obteniendo así la dirección más segura.

Este método calcula eficazmente la "profundidad" de la carretera en cada columna. Una columna que tiene muchos píxeles blancos antes de un píxel negro en la imagen representa un punto de la carretera que está más lejos. Al analizar desde abajo hacia arriba, se ignoran los píxeles blancos que puedan estar por encima de la línea de "no carretera", asegurando que solo se mide la carretera contigua al robot.

Una vez identificado el punto objetivo (colMasAlta), esta información se utiliza para calcular la velocidad lineal y angular del robot, consiguiendo que este se dirija hacia esta dirección. Para ello se establece una velocidad angular proporcional a la posición de la columna más alta (hacia la que el robot debe dirigirse) y una velocidad lineal inversamente proporcional a la angular.

Velocidad Angular:

El cálculo de la velocidad angular se realiza restándole al centro de la imagen la columna con más espacio en la carretera, para posicionarla en el centro. Este valor se normaliza dividiéndolo entre el centro de la imagen y se escala multiplicándolo por una constante (0.3).

$$\text{angularVelocity} = 0.4 \times ((imageWidth//2 - colMasAlta)/(imageWidth//2))$$

Velocidad Lineal:

La velocidad lineal no es constante, sino que se ajusta en función de la magnitud del giro requerido. Cuanto mayor sea el giro, menor será la velocidad

lineal, y viceversa (proporcionalidad inversa). Así, cuando el robot encuentra una zona segura donde tiene espacio se sobra, su velocidad lineal aumenta.

Finalmente realizamos un control para que el robot no se choque donde si todas las columnas que ve no superan el umbral de blancos ($blancosColMasAlta < 100$), el robot se para (velocidad lineal = 0) y gira hasta encontrar una salida.

Autómata del comportamiento de seguimiento de la carretera:
Estados:

- *Seguimiento normal*: el robot avanza manteniéndose centrado en la carretera mediante información visual.
- *Búsqueda de salida*: no se detecta suficiente carretera delante del robot y se realiza un giro exploratorio para localizar una zona transitable.

Transiciones:

- *Seguimiento normal* → *Búsqueda de salida*: el número máximo de píxeles blancos consecutivos en una columna es inferior al umbral establecido ($blancosColMasAlta < 100$).
- *Búsqueda de salida* → *Seguimiento normal*: detección de una columna válida con suficiente número de píxeles blancos que indica la presencia de carretera.

Variables de interés:

- `col_mas_alta`: columna de la imagen con mayor cantidad de píxeles blancos consecutivos, indicando la dirección más segura de la carretera.
- `blancosColMasAlta`: número de píxeles blancos consecutivos en `col_mas_alta`.
- `angularVelocity`: velocidad angular calculada para centrar la carretera en el eje del robot.
- `linearVelocity`: velocidad lineal ajustada en función del giro requerido y de la cantidad de carretera detectada.

2.1.2 Gestión de giros: Izquierda/Derecha

El subcomportamiento de giro es necesario para que el robot sea capaz de ejecutar el giro a izquierda o derecha en una intersección cuando se detecta la señal de tráfico correspondiente, inhibiendo de forma temporal el seguimiento normal de la carretera y asegurando que el giro se complete correctamente antes de retomar la conducción normal.

Para activar este comportamiento es necesario que la señal se detecte durante varios ciclos consecutivos (6). Una vez sucede esto, se guarda el ángulo inicial del giro del robot y se inhibe el seguimiento normal de la carretera.

Durante el giro, se modifica la imagen segmentada para anular visualmente tres cuartas partes del lado contrario del giro (se ponen todos esos píxeles en negro). Se tiene en cuenta la columna de más a la derecha o a la izquierda (dependiendo de hacia que lado sea el giro) y no se comienza a girar hasta que esta columna llegue a 124 blancos. Cuando esta condición se cumple se vuelve a aplicar la lógica de encontrar la columna con mayor número de blancos para centrar al robot hacia esa dirección.

Para determinar el fin del giro del robot, utilizamos una variable booleana que se pondrá a True cuando la diferencia del ángulo inicial del giro (el ángulo guardado al detectar la señal de giro) y el ángulo actual del robot sea 90º.

Una vez completado, se desactiva el comportamiento de giro y se vuelve al seguimiento normal de la carretera.

Por lo tanto, hemos decidido utilizar la odometría del robot para decidir cuando parar de girar. Además hemos segmentado la imagen para forzar la elección del carril correcto durante el giro.

Otra alternativa que probamos y descartamos fue empezar a girar al pasar un tiempo fijo desde la última vez que veía la señal. Esto lo aplicamos porque el robot dejaba de ver la señal mucho antes de llegar a la intersección por lo que nos interesaba mantener el comportamiento de seguimiento de la carretera algo más de tiempo. Sin embargo, fue imposible ajustar el tiempo de manera que se realizasen bien todos los giros del circuito ya que cada giro requería tiempos distintos debido a su naturaleza, a la posición del robot y a cómo había actuado anteriormente. Esto daba lugar a que en algunas ocasiones no girase a tiempo y, en otras, girase demasiado pronto, por lo que descartamos esta opción.

Autómata del comportamiento de giro (izquierda/derecha):
Estados:

- *Reposo*: el robot sigue la carretera utilizando el comportamiento de seguimiento normal.
- *Preparación del giro*: se detecta una señal de giro válida durante varios ciclos consecutivos y se inicializan las variables necesarias para la maniobra.
- *Girando*: el robot ejecuta el giro correspondiente empleando información visual y de odometría.

Transiciones:

- *Reposo* → *Preparación del giro*: detección consistente de una señal de giro (izquierda o derecha).
- *Preparación del giro* → *Girando*: almacenamiento del ángulo inicial del robot e inhibición del seguimiento de carretera.

- *Girando* → *Reposo*: la diferencia entre el ángulo actual y el ángulo inicial alcanza aproximadamente 90°.

Variables de interés:

- `senal_anterior`: tipo de señal detectada (LEFT/RIGHT) que activa el giro.
- `angulo_inicial_giro`: ángulo del robot al iniciar la maniobra de giro.
- `girando`: booleano que indica si el robot está actualmente girando.
- `seguir_girando`: booleano que mantiene activo el giro hasta completar la maniobra.

2.1.3 STOP

Cuando el robot detecta la señal de STOP, es necesario que se detenga completamente antes de una intersección, permanezca inmóvil durante un tiempo determinado (5 segundos) y, una vez cumplida la detención obligatoria, reanude la marcha de forma segura. Este comportamiento tiene máxima prioridad dentro de la arquitectura de control, ya que está relacionado con la seguridad.

Este subcomportamiento también se activa tras visualizar la señal 6 veces, y lo que se hace es registrar el instante inicial de la detención. Las velocidades del robot se ponen a cero, y se mantienen así hasta que pasa el tiempo obligatorio. Una vez hecho esto, ponemos a True la variable booleana que nos indica que el stop se ha realizado, y el robot vuelve al comportamiento de seguir la carretera.

En este caso no hemos conseguido que el robot realice el stop en el lugar que debería y lo hace demasiado pronto. Nuestra idea inicial, como en el caso de los giros, también consistía en esperar un tiempo desde que el robot dejase de ver la señal, ya que es necesario parar en la intersección, no cuando se ve la señal. Sin embargo, nos encontramos con el mismo problema que en el caso del giro: dependiendo de la trayectoria anterior del robot y del momento en el que dejase de ver la señal, realizaba el stop demasiado pronto o demasiado tarde.

Autómata del comportamiento STOP:
Estados:

- *Reposo*: el robot circula normalmente utilizando el comportamiento de seguimiento de la carretera.
- *Detenido*: el robot permanece completamente inmóvil durante el tiempo obligatorio de parada.

Transiciones:

- *Reposo* → *Detenido*: detección consistente de una señal de STOP durante varios ciclos consecutivos.

- *Detenido* → *Reposo*: transcurridos 5 segundos desde el inicio de la detención obligatoria.

Variables de interés:

- `stop_hecho`: booleano que indica si la detención por la señal STOP ha sido completada.
- `tiempo_inicial_stop`: instante en el que se detecta la señal STOP y se inicia la detención obligatoria.

2.2 Coordinación entre Subcomportamientos

Como ya hemos comentado, la estrategia de control del vehículo autónomo se ha basado en la descomposición del problema en varios subcomportamientos independientes, cada uno de ellos responsable de una tarea concreta. Para coordinar dichos comportamientos y garantizar un funcionamiento seguro y coherente del sistema, se ha implementado una arquitectura de subsunción, en la que los distintos módulos se organizan jerárquicamente según su prioridad.

En esta arquitectura, todos los comportamientos se ejecutan de forma continua, pero únicamente uno de ellos tiene el control efectivo del robot en cada instante. La selección del comportamiento activo se realiza en función del estado del entorno y de una serie de variables internas que representan el contexto actual del sistema.

Organización jerárquica de los comportamientos:

Se han definido tres niveles de prioridad:

1. Prioridad alta: comportamiento de STOP. Es el comportamiento más crítico desde el punto de vista de la seguridad. Cuando se detecta una señal de STOP, este módulo inhibe al resto de comportamientos y obliga al robot a detenerse completamente durante un tiempo determinado.

2. Prioridad media: comportamiento de giro. Se activa cuando se detecta una señal de giro (izquierda o derecha). Durante la ejecución del giro, el robot ignora las órdenes del seguimiento de carretera y utiliza únicamente las velocidades generadas por este comportamiento hasta completar la maniobra.

3. Prioridad baja: seguimiento de la carretera. Es el comportamiento por defecto. Se ejecuta cuando no existe ninguna señal de STOP ni un giro activo, manteniendo el robot centrado en la carretera mediante información visual.

Esta jerarquía permite que los comportamientos más importantes puedan interrumpir de forma inmediata a los de menor prioridad, garantizando una respuesta rápida ante situaciones críticas.

Mecanismo de coordinación:

La coordinación se implementa mediante un módulo de control global que evalúa periódicamente el estado de cada comportamiento. En cada ciclo de control, el sistema comprueba, por orden de prioridad, si alguno de los comportamientos de mayor nivel debe tomar el control del robot. En caso afirmativo, las velocidades generadas por dicho comportamiento sustituyen a las de los niveles inferiores.

De esta forma, el sistema evita la combinación directa de velocidades procedentes de distintos comportamientos y asegura que solo uno de ellos sea responsable del movimiento del robot en cada instante.

Gracias a esta organización, cada comportamiento puede desarrollarse y ajustarse de manera independiente, sin afectar al funcionamiento del resto. Además, la incorporación de nuevos comportamientos (por ejemplo, evasión de obstáculos) sería sencilla, bastando con asignarles un nivel de prioridad adecuado dentro de la arquitectura existente.

En conjunto, la coordinación entre comportamientos permite implementar una estrategia de control robusta, en la que el robot es capaz de seguir la carretera de forma continua, reaccionar adecuadamente ante intersecciones y detenerse con seguridad cuando se detecta una señal de STOP.

3 Pseudocódigo del nodo

En esta sección se detalla la lógica algorítmica del nodo de conducción autónoma. El sistema se basa en una arquitectura de subsunción que coordina la percepción visual, la detección de señales y el control de actuadores.

3.1 Gestión de Percepción y Señalización

Esta función es responsable de procesar las señales detectadas por el sistema de visión. Para garantizar la robustez, implementa un filtro de persistencia que requiere que una señal sea detectada en seis ciclos consecutivos antes de activar un cambio de estado en el robot.

Algoritmo 1: Filtro de persistencia para señales de tráfico.

```
Función traffic_signs_callback(msg):
    if msg.signs then
        max_sign ← Seleccionar señal con mayor BBox
        sign_lbl ← clase de max_sign
    else
        sign_lbl ← EMPTY

    if sign_lbl ≠ EMPTY y senal_anterior == EMPTY then
        contador_sign_vista ← contador_sign_vista + 1
        if contador_sign_vista > 6 then
            // Confirmación de la señal detectada
            if sign_lbl == STOP then
                tiempo_inicial_stop ← tiempoActual()
                stop_hecho ← False
            else if sign_lbl ∈ {LEFT, RIGHT} then
                seguir_girando ← True
                angulo_inicial_giro ← pose.yaw
            senal_anterior ← sign_lbl
            contador_sign_vista ← 0
```

3.2 Extracción de la Referencia Visual

Es la función encargada de analizar la carretera segmentada. Calcula la "profundidad" de la vía analizando cada columna de píxeles desde la parte inferior hasta encontrar el primer obstáculo (píxel negro), identificando así la dirección con mayor espacio libre.

Algoritmo 2: Cálculo de la columna de máxima profundidad.

```
Función get_col_ref(cv_image):
    mask_blanco ← píxeles color (255, 255, 255)
    mask_negro ← píxeles color (0, 0, 0)
    for cada columna i do
        Analizar columna desde abajo hacia arriba
        negros_idx ← índice del primer píxel negro
        blancos_col_counts[i] ← suma de blancos antes del límite
    col_mas_alta ← argmax(blancos_col_counts)
    return col_mas_alta, blancos_col_derecha, blancos_col_izquierda
```

3.3 Procesamiento de Imagen y Control de Giro

Esta función gestiona la vista del robot durante las maniobras de intersección. Aplica máscaras para ocultar el carril contrario y utiliza los umbrales de píxeles

laterales para determinar el momento exacto en el que el robot debe comenzar a centrarse en el nuevo carril.

Algoritmo 3: Control visual de velocidad angular y lineal durante giros.

```
Función ground_image_callback(msg):
    cv_image ← Convertir ROS Image a OpenCV
    if seguir_girando then
        // Máscara dinámica de carril
        if senal_anterior == LEFT then
            cv_image[:, ancho/4 : ] ← Negro
        else if senal_anterior == RIGHT then
            cv_image[:, : 3*ancho/4] ← Negro
        col_mas_alta, blD, blI ← get_col_ref(cv_image)
        if blD > 124 o blI > 124 o girando then
            girando ← True
            ω ← 0.4 × ((ancho/2 − col_mas_alta)/(ancho/2))
            v ← (|ω| > 0.1)?(0.05 × |ω|) : 0.2
            giro_vel ← (v, ω)
```

3.4 Arquitectura de Subsumción (Coordinador Central)

Es el núcleo del sistema de control. Se ejecuta periódicamente para decidir qué comando de velocidad debe publicarse basándose en una jerarquía estricta: el STOP tiene la prioridad más alta, seguido de las maniobras de giro y, finalmente, el seguimiento de carretera estándar.

Algoritmo 4: Coordinador de estados por subsunción.

```
Función subsumption_architecture():
    if senal_anterior == STOP y ¬stop_hecho then
        // Prioridad Máxima: STOP
        (v, ω) ← (0.0, 0.0)
        if TiempoActual - tiempo_inicial_stop ≥ 5.0 then
            stop_hecho ← True; senal_anterior ← EMPTY
    else if seguir_girando then
        // Prioridad Media: Giro Activo
        (v, ω) ← giro_vel
        if |pose.yaw − angulo_inicial_giro| > 1.57 then
            seguir_girando ← False; girando ← False
            senal_anterior ← EMPTY
    else
        // Prioridad Baja: Seguimiento Normal
        (v, ω) ← road_following_vel
    Publicar (v, ω) en topic /cmd_vel
```

3.5 Gestión de Odometría y Sensores

Finalmente, estas funciones se encargan de actualizar la telemetría del robot. La odometría es esencial para calcular el ángulo de 90 grados (1.57 radianes) necesario para completar los giros de forma precisa.

Algoritmo 5: Callbacks de actualización de estado y sensores.

```
Función odom_callback(msg):
    (roll, pitch, yaw) ← euler_from_quaternion(msg.orientation)
    pose ← (x, y, yaw)

Función lidar_callback(msg):
    front_scan ← Filtrar rango frontal (180º)
    Gestionar lecturas según tipo de sensor (s2/a1)
```

4 Análisis del comportamiento

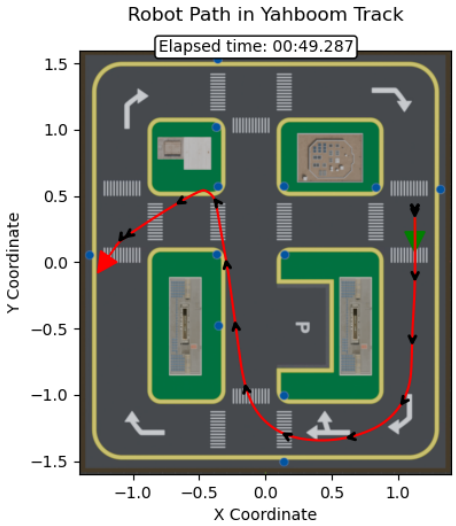

Figure 1: Recorrido del robot con umbral de blancos necesarios para girar 122

El umbral de blancos que el robot necesita en la columna de la izquierda o derecha de la carretera para empezar a girar que mejor ha funcionado es un umbral de 122. De esta manera el primer giro en la señal se realiza de manera correcta y, en el siguiente giro, comienza a girar algo tarde por lo que se choca con una señal. Finalmente consigue realizar ese giro pero no le da tiempo de ver la siguiente señal 6 veces seguidas (umbral necesario para realizar la instrucción que indica esta) y sigue la carretera en dirección contraria.

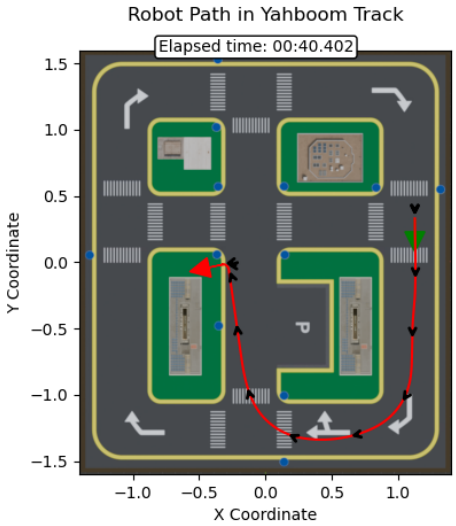

Figure 2: Recorrido del robot con umbral de blancos necesarios para girar menor a 122

Cuando se establece un umbral de blancos en la columna de la izquierda o derecha para girar menor a 122, el robot sigue haciendo la primera señal bien pero en la segunda gira demasiado pronto, por lo que se choca contra el edificio (aulario) y ya no puede seguir realizando el circuito.

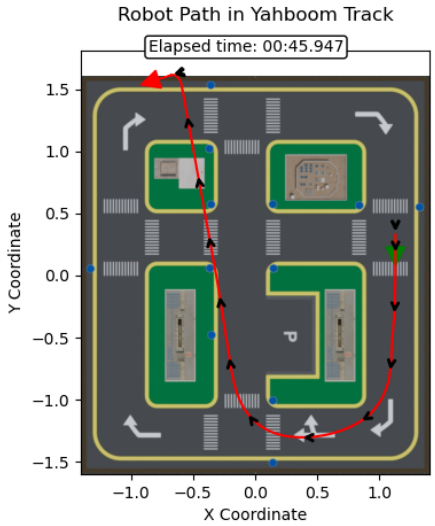

Figure 3: Recorrido del robot con umbral de blancos necesarios para girar mayor a 122

Por el contrario, cuando se establece un umbral de blancos para el giro mayor de 122, el robot también sigue realizando el primer giro bien, pero en la segunda señal sólo cambia su trayectoria algo hacia la izquierda. Esto hace que no llegue a girar del todo y no realice el giro en el cruce, pero que tampoco pueda ver correctamente la siguiente señal y por ello se desvíe en dirección contraria en el cruce.

Con esta última opción, con el robot real se consiguió que realizase la primera señal, ignorase la segunda, y realizase correctamente la tercera y la cuarta ya que no se desvió tanto y consiguió ver las señales. Después, siguió recto hasta la pared, lo cual no tiene una explicación clara ya que, aunque no vea señales, este no es el comportamiento que le ordena roadFollowing. Esto se muestra en el video adjunto en la entrega.

5 Conclusiones

Finalmente, no hemos sido capaces de conseguir que el robot tenga un comportamiento adecuado de conducción autónoma, ya que mediante las diferentes estrategias que hemos probado, este nunca conseguía pasar de la segunda señal. Sí que hemos conseguido un comportamiento adecuado para el seguimiento de la carretera y para la primera señal, pero al depender de variables fijas que dependen de la visión del robot en el momento y del comportamiento anterior

que este ha tenido, no sirven los mismos hiperparámetros para todos los cruces. Como hemos explicado en este informe, hemos sido conscientes de los problemas que estaban ocurriendo y del porqué de estos. No obstante, no hemos conseguido idear ninguna estrategia que los solvente ya que todo lo que se nos ocurría tenía dependencias y hacía que el comportamiento fuese el mismo o peor.